E-Commerce Architecture • Frontend Integration Guide

دليل واجهات برمجة التطبيقات (API) لمطوري الواجهة الأمامية

مرجع شامل ومفصل لجميع نقاط النهاية (Endpoints)، قواعد المصادقة، هياكل البيانات، كواليس المخزون اللحظي عبر gRPC، وآلية إنشاء الطلبات الآمنة.

الواجهة الأساسية (Base URL):	http://localhost:8000/api
نظام المصادقة:	JWT Bearer Token
صيغة البيانات:	application/json
إصدار التوثيق:	1.0.0 (October 2026)

1. إرشادات عامة وقواعد التكامل للفرونت إند

قبل البدء في استدعاء الـ APIs، يرجى الالتزام بالمعايير الهندسية التالية لضمان تجربة مستخدم سلسة وعالية الأمان:

📌 **الترويسات العامة المطلوبة (General Headers)**

يجب إرسال هذه الترويسات مع كافة طلبات الـ HTTP لضمان معالجة وتنسيق الردود بصيغة JSON:

```
Accept: application/json
Content-Type: application/json
```

🔐 **ترويسة المصادقة (Authentication Header)**

لجميع المسارات المحمية، يتم إرسال التوكن في الترويسة بالشكل التالي:

```
Authorization: Bearer <access_token>
```

هيكل ترقيم الصفحات الموحد (Pagination Structure)

جميع الطلبات التي تُرجع قوائم متعددة العناصر تتبع الهيكل القياسي التالي لـ Laravel Resources:

```
{
  "data": [ /* مصفوفة النتائج */ ],
  "links": {
    "first": "http://localhost:8000/api/...?page=1",
    "last": "http://localhost:8000/api/...?page=5",
    "prev": null,
    "next": "http://localhost:8000/api/...?page=2"
  },
  "meta": {
    "current_page": 1,
    "from": 1,
    "last_page": 5,
    "per_page": 15,
    "to": 15,
    "total": 72
  }
}
```

2. نظام المصادقة والحسابات (Authentication)

يعتمد النظام على **JSON Web Tokens (JWT)** مع فترة صلاحية **60 دقيقة** لكل رمز.

POST `/api/register` عام (Public)

تسجيل مستخدم جديد بدور **عميل (customer)**. يتم تسجيل دخوله فورًا وإرجاع رمز JWT في الاستجابة.

> **قاعدة أمنية:** يتم تعيين دور المستخدم تلقائيًا على `customer` في السيرفر، ولا يُقبل حقل `role` من المدخلات لمنع تصعيد الصلاحيات.

الحقل	النوع	الحالة	الوصف والقيود
`name`	string	إجباري	الاسم الكامل للمستخدم (بحد أقصى 255 حرف)
`email`	string	إجباري	بريد إلكتروني صالح وفريد (غير مسجل مسبقًا)
`password`	string	إجباري	كلمة المرور (8 أحرف على الأقل)
`password_confirmation`	string	إجباري	تأكيد كلمة المرور، ويجب أن يطابق password تمامًا

الاستجابة الناجحة (CREATED 201):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "expires_in": 3600,
  "user": {
    "id": 1,
    "name": "أحمد محمود",
    "email": "ahmed@example.com",
    "role": "customer",
    "created_at": "2026-10-28T10:00:00.000000Z"
  }
}
```

POST `/api/login` عام (Public)

تسجيل الدخول باستخدام البريد الإلكتروني وكلمة المرور للحصول على رمز JWT. معدل الطلبات محدد بـ (10 طلبات في الدقيقة).

الحقل	النوع	الحالة	الوصف
`email`	string	إجباري	البريد الإلكتروني المسجل
`password`	string	إجباري	كلمة المرور

الاستجابة الناجحة (OK 200):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "expires_in": 3600,
  "user": {
    "id": 1,
    "name": "أحمد محمود",
    "email": "ahmed@example.com",
    "role": "customer",
    "created_at": "2026-10-28T10:00:00.000000Z"
  }
}
```

استجابة بيانات خاطئة (UNAUTHORIZED 401):

```
{ "message": "Invalid credentials." }
```

GET `/api/me` مسجل الدخول (Customer/Admin)

جلب الملف الشخصي للمستخدم صاحب الجلسة الحالية والتحقق من دوره (role).

الاستجابة الناجحة (OK 200):

```
{
  "data": {
    "id": 1,
    "name": "أحمد محمود",
    "email": "ahmed@example.com",
    "role": "customer",
    "created_at": "2026-10-28T10:00:00.000000Z"
  }
}
```

POST `/api/logout` مسجل الدخول (Customer/Admin)

إبطال الرمز الحالي في الخادم وإنهاء الجلسة.

الاستجابة الناجحة (OK 200):

```
{ "message": "Logged out successfully." }
```

3. التصنيفات (Categories)

GET /api/categories — عام (Public)

استعراض كافة التصنيفات مرتبة أبجديًا ومقسمة إلى صفحات (30 تصنيف لكل صفحة).

معامل الاستعلام (Query Param)	النوع	الافتراضي	الوصف
page	integer	1	رقم الصفحة المطلوبة

الاستجابة الناجحة (200 OK):

```
{
  "data": [
    {
      "id": 1,
      "name": "هواتف ذكية",
      "description": "أحدث الهواتف الذكية والإلكترونيات",
      "created_at": "2026-10-28T10:00:00.000000Z",
      "updated_at": "2026-10-28T10:00:00.000000Z"
    }
  ],
  "links": { /* ... */ },
  "meta": { /* ... */ }
}
```

GET /api/categories/{category} — عام (Public)

عرض بيانات تصنيف محدد بواسطة معرّفه الرقمي (ID).

POST /api/categories — مشرف فقط (Admin)

إضافة تصنيف جديد. الحقول: `name` (إجباري)، `description` (اختياري). الرد: `201 Created`.

PUT /api/categories/{category} — مشرف فقط (Admin)

تعديل بيانات التصنيف. الرد: `200 OK`.

DELETE `/api/categories/{category}` مشرف فقط (Admin)

حذف تصنيف محدد.

قاعدة حماية البيانات: إذا كان التصنيف يحتوي على منتجات مرتبطة به، يرفض السيرفر الحذف ويعيد: `409 Conflict` : `{ ".message": "Categories containing products cannot be deleted" }`

4. الكتالوج، المنتجات، والمخزون الحي (Products & Inventory)

GET `/api/products` عام (Public)

استعراض المنتجات النشطة مع دعم البحث النصي، الفلترة حسب التصنيف، والترقيم.

المعامل (Query Param)	النوع	الافتراضي	الوصف
`search`	string	فارغ	بحث باسم المنتج (مطابقة جزئية)
`category_id`	integer	فارغ	تصفية المنتجات حسب تصنيف معين
`per_page`	integer	15	عدد المنتجات في الصفحة (الأدنى 1، الأقصى 100)
`page`	integer	1	رقم الصفحة

الاستجابة الناجحة (OK 200):


```
{
  "data": [
    {
      "id": 1001,
      "name": "آيفون 15 برو",
      "description": "هاتف بتصميم من التيتانيوم",
      "price": "999.00",
      "image_url": "http://localhost:8000/storage/products/iphone15.jpg",
      "category": {
        "id": 1,
        "name": "هواتف ذكية"
      },
      "is_active": true,
      "created_at": "2026-10-28T10:00:00.000000Z"
    }
  ],
  "links": { /* ... */ },
  "meta": { /* ... */ }
}
```

GET `/api/products/{product}` عام (Public)

جلب التفاصيل الكاملة للمنتج متضمنة حقل **المخزون المتاح لحظيًا (stock)**.

> 💡 **كواليس الاتصال بـ gRPC:**
> يقوم خادم Laravel باستدعاء مباشر عبر بروتوكول gRPC لخدمة `inventory-service` عبر دالة `GetStock(productId)` لجلب الكمية المتاحة حاليًا (المخزون الكلي - الكميات المحجوزة).

الاستجابة الناجحة (OK 200):

```
{
  "data": {
    "id": 1001,
    "name": "آيفون 15 برو",
    "description": "هاتف بتصميم من التيتانيوم",
    "price": "999.00",
    "image_url": "http://localhost:8000/storage/products/iphone15.jpg",
    "category": { "id": 1, "name": "هواتف ذكية" },
    "is_active": true,
    "stock": 48,
    "created_at": "2026-10-28T10:00:00.000000Z"
  }
}
```

> **حالة استثنائية (Service Unavailable 503):** إذا تعذر الاتصال بخدمة المخزون، يُرجع السيرفر:
>
> `{ ""INVENTORY_UNAVAILABLE" "code":", "message": "Current inventory is unavailable. Please try again." }`

POST `/api/products` مشرف فقط (Admin)

إضافة منتج جديد. يدعم رفع الصورة عبر `multipart/form-data`.

الحقل	النوع	الحالة	الوصف
`name`	string	إجباري	اسم المنتج
`price`	numeric	إجباري	السعر (الحد الأدنى 0.01)
`category_id`	integer	إجباري	معرّف التصنيف التابع له
`image`	file	اختياري	ملف صورة صالح (بحد أقصى 2MB)
`is_active`	boolean	اختياري	حالة التنشيط (الافتراضي true)

DELETE `/api/products/{product}` مشرف فقط (Admin)

إلغاء تنشيط المنتج (Soft Delete). لن يظهر في المتجر للعملاء مع بقائه محفوظاً للطلبات القديمة.

5. نظام الطلبات وحجز المخزون (Orders & Saga Pattern)

POST `/api/orders` مسجل الدخول (Customer)

إنشاء طلب شراء جديد وقفل المخزون المقابل. هذه العملية محكومة بنمط المعاملات الموزعة والتراجع الذاتي (Durable Compensation).

⚠️ **ترويسة Idempotency-Key الإجبارية:**

يجب على الفرونت إند توليد UUID v4 جديد لكل محاولة طلب، وإرساله في ترويسة الـ Header:

```
Idempotency-Key: 9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d
```

لماذا؟ لمنع تكرار خصم المخزون أو إنشاء طلبين بالخطأ إذا قام المستخدم بالضغط مرتين أو انقطع الاتصال ثم أُعيدت المحاولة.

قاعدة أمان الأسعار: لا يرسل الفرونت إند أي أسعار أو إجماليات! السيرفر يقرأ الأسعار المعتمدة من قاعدة البيانات ويحسب الإجمالي بدقة `bcmath`.

جسم الطلب (REQUEST BODY):

```
{
  "items": [
    {
      "product_id": 1001,
      "quantity": 2
    }
  ]
}
```

الاستجابة الناجحة (CREATED 201):

```
{
  "data": {
    "id": 105,
    "user_id": 1,
    "total": "1998.00",
    "status": "pending",
    "items": [
      {
        "id": 1,
        "quantity": 2,
        "unit_price": "999.00",
        "subtotal": "1998.00",
        "product": {
          "id": 1001,
          "name": "آيفون 15 برو",
          "price": "999.00"
        }
      }
    ],
    "created_at": "2026-10-28T10:30:00.000000Z",
    "updated_at": "2026-10-28T10:30:00.000000Z"
```

```
  }
}
```

كود الخطأ	كود الاستجابة الداخلي	معنى الخطأ والإجراء المطلوب في الفرونت إند
`409 Conflict`	`INSUFFICIENT_STOCK`	الكمية غير متوفرة. أظهر تنبيه للمستخدم بنفاد الكمية لتعديل السلة.
`503 Unavailable`	`RESERVATION_OUTCOME_UNKNOWN`	انتهت مهلة التأكيد ونتيجة الحجز قيد التحقق. **أعد المحاولة بنفس الـ Idempotency-Key حصراً** بعد ثانية.
`422 Unprocessable`	-	مفتاح Idempotency مفقود أو مصفوفة items فارغة أو معرّف منتج غير نشط.

GET `/api/orders` مسجل الدخول (Customer)

استعراض سجل طلبات المستخدم الحالي مرتبة من الأحدث إلى الأقدم (15 طلب في الصفحة).

GET `/api/orders/{order}` صاحب الطلب أو المشرف فقط

عرض تفاصيل طلب محدد مع بنوده والمنتجات التابعة له. إذا حاول مستخدم فتح طلب لعميل آخر، يُرجع السيرفر `404 Not Found` لحماية الخصوصية.

PATCH `/api/admin/orders/{order}/status` مشرف فقط (Admin)

تحديث حالة الطلب بواسطة المشرف وفق شجرة تحولات الحالة الصارمة (State Machine).

```
pending ──► confirmed ──► processing ──► shipped ──► delivered (نهائي)
              │              │             │
              └──► cancelled └──► cancelled └──► cancelled (نهائي)
```

> **تحرير المخزون التلقائي:** عند تحويل الطلب إلى `cancelled`، يتصل الخادم فورًا بخدمة المخزون لتحرير الكميات المحجوزة وإرجاعها للبيع تلقائيًا!

جسم الطلب (REQUEST BODY):

```
{ "status": "processing" }
```

6. الإشعارات (Notifications)

GET `/api/notifications` مسجل الدخول (Customer)

استعراض إشعارات العميل (مثل إشعارات إنشاء الطلب وتحديث حالته)، مقسمة لصفحات (15 إشعار بالصفحة).

الاستجابة الناجحة (OK 200):


```
{
  "data": [
    {
      "id": "c4b9d031-7bfa-4c8d-8a5e-859e4b600f12",
      "type": "App\\Notifications\\OrderNotification",
      "data": {
        "order_id": 105,
        "status": "confirmed",
        "total": "1998.00",
        "message": "Your order #105 status changed to confirmed."
      },
      "read_at": null,
      "created_at": "2026-10-28T10:35:00.000000Z"
    }
  ],
  "links": { /* ... */ },
  "meta": { /* ... */ }
}
```


PATCH `/api/notifications/read-all` مسجل الدخول

تحديد كافة الإشعارات غير المقروءة كمقروءة دفعة واحدة. الرد: { "message": "Notifications marked as read." }.

PATCH `/api/notifications/{notification}/read` مسجل الدخول

تحديد إشعار محدد كمقروء بواسطة معرّفه الفريد (UUID). الرد: { "message": "Notification marked as read." }.

7. أمثلة كود التكامل للواجهة الأمامية (Frontend Code Examples)

1. إعداد عميل Axios مع حقن التوكن ومعالجة انتهاء الجلسة

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:8000/api',
  headers: {
    'Accept': 'application/json',
    'Content-Type': 'application/json',
  },
});

// حقن توكن الـ JWT في كل طلب
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('access_token');
  if (token) {
    config.headers.Authorization = "Bearer " + token;
  }
  return config;
});

// التعامل التلقائي مع انتهاء صلاحية التوكن (401)
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('access_token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default api;
```

2. دالة إرسال الطلب مع Idempotency-Key وإعادة المحاولة التلقائية

```
import { v4 as uuidv4 } from 'uuid';
import api from './api';

export async function submitOrder(cartItems) {
  // توليد مفتاح فريد لهذه المحاولة
  const idempotencyKey = uuidv4();

  const payload = {
    items: cartItems.map(item => ({
      product_id: item.productId,
      quantity: item.quantity,
    })),
  };

  try {
```

```
    const res = await api.post('/orders', payload, {
      headers: { 'Idempotency-Key': idempotencyKey },
    });
    return res.data.data;
  } catch (err) {
    // إذا كانت النتيجة غير مؤكدة، نعيد المحاولة بنفس المفتاح حصراً
    if (err.response?.data?.code === 'RESERVATION_OUTCOME_UNKNOWN') {
      console.warn('Retrying with identical Idempotency-Key...');
      await new Promise(r => setTimeout(r, 1500));
      const retryRes = await api.post('/orders', payload, {
        headers: { 'Idempotency-Key': idempotencyKey },
      });
      return retryRes.data.data;
    }
    throw err;
  }
}
```